

Lab4-Pytorch+CNN

本次实验围绕深度学习在图像任务中的应用展开，包含两个核心模块：一是基于 ResNet20 模型对 CIFAR-10 数据集进行分类，通过优化学习率与训练策略提升模型性能；二是利用 Pytorch 官方预训练模型提取图像特征，结合特征向量夹角计算实现图像检索。实验结果表明，CIFAR-10 分类任务的测试集准确率可达 86% 以上，ResNet50 与 VGG16 在图像检索任务中表现更优，为深度学习在图像分类与检索领域的应用提供了实践参考。

1 实验概览

1.1 CIFAR-10 数据集分类

本实验的核心目标是利用深度学习技术对 CIFAR-10 数据集完成分类任务。实验选用 ResNet20 作为基础模型，通过调整学习率、控制训练轮次等关键训练策略，优化模型的收敛过程与分类性能，最终实现对 CIFAR-10 数据集中 10 类图像的高效区分。

1.2 使用 Pytorch 提取图像特征并检索

该实验旨在借助 Pytorch 框架提供的预训练深度模型，完成图像特征提取与检索任务。实验选用 VGG16、ResNet50、InceptionV3 三种经典预训练模型，先通过模型提取图像的特征向量，再通过计算不同图像特征向量间的夹角衡量相似度，最终实现“以图搜图”，即根据待检索图像从数据库中匹配出最相似的图像。

2 解决思路及关键步骤

2.1 CIFAR-10 数据集分类

2.1.1 代码功能补充

在实验所用的 exp2.py 文件中，重点补充了 test 函数的核心逻辑，使其具备计算模型在测试集上损失值与准确率的能力。该过程无需更新模型参数，仅通过模型输出结果

与测试集真实标签的对比，统计正确预测的样本数量，进而计算测试准确率，为模型性能评估提供关键指标。

```
1         for batch_idx, (inputs, targets) in enumerate(trainloader):
2             optimizer.zero_grad()
3             outputs = model(inputs)
4             # The outputs are of size [128x10].
5             # 128 is the number of images fed into the model
6             # (yes, we feed a certain number of images into the model at
              the same time,
7             # instead of one by one)
8             # For each image, its output is of length 10.
9             # Index i of the highest number suggests that the prediction
              is classes[i].
10            loss = criterion(outputs, targets)
11            loss.backward()
12            optimizer.step()
13            train_loss += loss.item()
14            _, predicted = outputs.max(1)
15            total += targets.size(0)
16            correct += predicted.eq(targets).sum().item()
17            print('Epoch [%d] Batch [%d/%d] Loss: %.3f | Traininig Acc:
              %.3f%% (%d/%d) '
18                  % (epoch, batch_idx + 1, len(trainloader), train_loss
                  / (batch_idx + 1),
19                     100. * correct / total, correct, total))
```

2.1.2 训练策略优化

为提升模型性能，实验对学习率与训练流程进行了针对性调整：

1. 学习率调整：将单一学习率变量转换为学习率列表（lrs），包含 0.1、0.1、0.01 三个关键值，实现分阶段学习率控制；
2. 训练轮次规划：总计完成 30 轮训练，具体分为两个阶段：第一阶段使用 0.1 的学习率训练 20 轮（分两次执行，每次训练 10 轮，每轮训练后均进行测试）；第二阶段使用 0.01 的学习率训练 10 轮（单次执行，每轮训练后同样进行测试）；
3. 训练流程控制：通过循环遍历学习率列表与训练轮次，依次执行 train 函数（模型训练）与 test 函数（性能测试），确保每一轮训练的效果都能被及时评估，最终使模型在测试集上的准确率稳定达到 86% 以上。

```
1     lrs = [0.1, 0.1, 0.01]
```

```

2     for i in range(len(lrs)):
3     for epoch in range(start_epoch, end_epoch + 1):
4         train(epoch, lr=lrs[i])
5         test(epoch)
6         train(epoch, lr=lrs[i])
7         test(epoch)

```

2.2 实验 5.2 使用 Pytorch 提取图像特征并检索

2.2.1 图像特征提取

实验通过三种预训练模型（VGG16、ResNet50、InceptionV3）完成图像特征提取，三种模型的特征提取流程一致，核心步骤如下：

1. 模型加载：从 Pytorch 官方仓库加载在 ImageNet 数据集上预训练完成的模型参数，确保模型具备成熟的特征提取能力；
2. 图像预处理：对输入图像执行统一的归一化操作（三种模型的归一化参数相同），并根据不同模型的输入要求调整图像尺寸（如 VGG16 与 ResNet50 需将图像调整为 224×224 ，InceptionV3 需调整为 299×299 ），再转换为模型可处理的张量格式；
3. 特征层选择：针对三种模型分别选取合适的特征输出层——VGG16 与 ResNet50 选取 avgpool 层，InceptionV3 选取 Mixed_7c 层，通过模拟官方示例方法从这些层提取图像的深层特征向量；
4. 特征保存：将提取的特征向量通过 numpy 库保存至指定路径（如 Feature_inception、Feature_vgg16、Feature_resnet50 目录），为后续相似度计算与检索提供数据基础。

```

1     print('Loading model: InceptionV3')
2     inception_model = torch.hub.load('pytorch/vision', 'inception_v3',
3         pretrained=True)
4
5     img_mean = [0.485, 0.456, 0.406]
6     img_std = [0.229, 0.224, 0.225]
7     normalizer = transforms.Normalize(mean=img_mean, std=img_std)
8
9     image_transforms = transforms.Compose([
10         transforms.Resize(299),
11         transforms.CenterCrop(299),
12         transforms.ToTensor(),
13         normalizer
14     ])
15     def get_inception_features(input_tensor):

```

```

16     feat = inception_model.Conv2d_1a_3x3(input_tensor)
17     feat = inception_model.Conv2d_2a_3x3(feat)
18     feat = inception_model.Conv2d_2b_3x3(feat)
19     feat = inception_model.Conv2d_3b_1x1(feat)
20     feat = inception_model.Conv2d_4a_3x3(feat)
21     feat = inception_model.Mixed_5b(feat)
22     feat = inception_model.Mixed_5c(feat)
23     feat = inception_model.Mixed_5d(feat)
24     feat = inception_model.Mixed_6a(feat)
25     feat = inception_model.Mixed_6b(feat)
26     feat = inception_model.Mixed_6c(feat)
27     feat = inception_model.Mixed_6d(feat)
28     feat = inception_model.Mixed_6e(feat)
29     feat = inception_model.Mixed_7a(feat)
30     feat = inception_model.Mixed_7b(feat)
31     feat = inception_model.Mixed_7c(feat)
32     return feat
33
34
35 def process_and_save_features(img_path, output_path):
36     print('Preparing image data...')
37     img = default_loader(img_path)
38     processed_img = image_transforms(img)
39     img_tensor = torch.unsqueeze(processed_img, dim=0)
40
41     print('Extracting features...')
42     start_time = time.time()
43     features = get_inception_features(img_tensor)
44     features_np = features.detach().cpu().numpy()
45     elapsed_time = time.time() - start_time
46     print(f'Feature extraction time: {elapsed_time:.2f} seconds')
47
48     print('Saving features...')
49     np.save(output_path, features_np)

```

2.2.2 相似度计算与图像检索

图像检索任务围绕“特征匹配”展开，核心步骤包括：

1. 数据集特征预处理：通过上述特征提取方法，对数据库 Dataset 目录中的所有

图像（共 50 张）进行特征提取，并将对应的特征向量分别保存至三种模型对应的特征目录中；

2. 相似度计算：定义 `cal_similarity` 函数计算两个特征向量的相似度：首先通过 `flatten()` 函数将特征向量展平，再利用 `np.dot` 计算向量点积，结合 `np.linalg.norm` 计算向量模长，最终通过夹角公式将点积与模长转换为夹角值（夹角越小，相似度越高）；

3. 检索流程执行：- 对待检索图像（来自 `Query_image` 目录），先通过三种模型分别提取特征并保存至 `Query_feature` 目录；

- 定义 `query_k_image()` 函数，计算待检索图像特征与数据库中所有图像特征的夹角，筛选出夹角最小的前 `k` 张图像（实验中 `k=5`），返回其标签与夹角值；

- 利用 `matplotlib` 库将待检索图像与 Top-5 相似图像进行可视化展示，并将结果保存至 `Output` 目录，完成检索结果的直观呈现。

```
1 DATASET_DIR = "Dataset"
2 FEAT_DIRS = {
3     "ResNet50": "Feature_resnet50",
4     "VGG16": "Feature_vgg16",
5     "InceptionV3": "Feature_inceptionv3"
6 }
7 QUERY_IMAGE_DIR = "Query_image"
8 QUERY_FEAT_DIR = "Query_feature"
9 OUTPUT_DIR = "Output"
10
11
12 def compute_angle_similarity(feat_a, feat_b):
13
14     vec_a = feat_a.flatten()
15     vec_b = feat_b.flatten()
16     cos_sim = np.dot(vec_a, vec_b) / (np.linalg.norm(vec_a) * np.
17         linalg.norm(vec_b))
18     angle_sim = np.arccos(cos_sim) / np.pi * 180
19     return angle_sim
20
21 def batch_extract_features(model_extractor, data_dir, save_dir,
22     img_count=50):
23
24     for img_idx in range(1, img_count + 1):
25         img_path = os.path.join(data_dir, f"{img_idx}.jpg")
26         feat_save_path = os.path.join(save_dir, f"{img_idx}.npy")
27         model_extractor(img_path, feat_save_path)
```

```

27
28
29 def batch_extract_resnet50(data_dir, save_dir, img_count=50):
30     for img_idx in range(1, img_count + 1):
31         img_path = os.path.join(data_dir, f"{img_idx}.jpg")
32         feat_save_path = os.path.join(save_dir, f"{img_idx}.npy")
33
34         test_image = extract_feature.default_loader(img_path)
35         input_image = extract_feature.trans(test_image)
36         input_image = torch.unsqueeze(input_image, 0) # 现在 torch
                 已导入, 可正常调用
37         image_feature = extract_feature.features(input_image)
38         image_feature = image_feature.detach().numpy()
39         np.save(feat_save_path, image_feature)
40
41 def batch_extract_vgg16(data_dir, save_dir, img_count=50):
42     from vgg16 import process_image_and_save_features # 导入正确的
                 VGG16特征保存函数
43     for img_idx in range(1, img_count + 1):
44         img_path = os.path.join(data_dir, f"{img_idx}.jpg")
45         feat_save_path = os.path.join(save_dir, f"{img_idx}.npy")
46         process_image_and_save_features(img_path, feat_save_path) #
                 调用带路径参数的函数
47
48 def batch_extract_inception(data_dir, save_dir, img_count=50):
49     from inception import process_and_save_features # 导入正确的
                 inception特征保存函数
50     for img_idx in range(1, img_count + 1):
51         img_path = os.path.join(data_dir, f"{img_idx}.jpg")
52         feat_save_path = os.path.join(save_dir, f"{img_idx}.npy")
53         process_and_save_features(img_path, feat_save_path) # 调用
                 带路径参数的函数
54
55 def find_top_similar(query_feat_path, feat_dir, top_k=5):
56
57     query_feat = np.load(query_feat_path)
58     all_similarities = []
59     all_indices = []
60

```

```

61     for img_idx in range(1, 51):
62         feat_path = os.path.join(feat_dir, f"{img_idx}.npz")
63         try:
64             dataset_feat = np.load(feat_path)
65         except FileNotFoundError:
66             print(f"Warning: Feature file for index {img_idx} not found")
67             continue
68
69         sim_score = compute_angle_similarity(query_feat,
70                                             dataset_feat)
71         all_similarities.append(sim_score)
72         all_indices.append(img_idx)
73
74     sim_array = np.array(all_similarities)
75     idx_array = np.array(all_indices)
76     sorted_idx = np.argsort(sim_array)[:top_k]
77
78     return idx_array[sorted_idx], sim_array[sorted_idx]
79
80 def plot_similar_results(query_img_path, top_indices,
81                          top_similarities, model_name, save_path):
82
83     fig, axes = plt.subplots(1, 6, figsize=(28, 5))
84     fig.suptitle(model_name, fontsize=16)
85
86     query_img = plt.imread(query_img_path)
87     axes[0].imshow(query_img)
88     axes[0].axis('off')
89     axes[0].set_title('Query Image')
90
91     for idx, (img_idx, sim_score) in enumerate(zip(top_indices,
92                                                    top_similarities)):
93         img_path = os.path.join('./Dataset', f"{img_idx}.jpg")
94         similar_img = plt.imread(img_path)
95         axes[idx + 1].imshow(similar_img)
96         axes[idx + 1].axis('off')
97         axes[idx + 1].set_title(f'Top {idx+1} \n Index: {img_idx} \

```

```

nSimilarity:_{sim_score:.2f}°')
96
97 plt.tight_layout()
98 plt.savefig(save_path, bbox_inches='tight')
99 plt.close()
100
101
102 def main():
103     DATASET_DIR = './Dataset'
104     OUTPUT_DIR = './Output'
105     FEAT_DIRS = {
106         'ResNet50': './Feature_resnet50',
107         'VGG16': './Feature_vgg16',
108         'InceptionV3': './Feature_inception'
109     }
110     QUERY_FEAT_DIR = './Query_feature'
111     QUERY_IMG_DIR = './Query_image'
112     QUERY_IMG_COUNT = 3
113
114     os.makedirs(QUERY_FEAT_DIR, exist_ok=True)
115     os.makedirs(OUTPUT_DIR, exist_ok=True)
116     for dir_path in FEAT_DIRS.values():
117         os.makedirs(dir_path, exist_ok=True)
118     for query_idx in range(QUERY_IMG_COUNT):
119
120         query_img_path = os.path.join(QUERY_IMG_DIR, f"{query_idx+
121             1}.jpg")
122
123         if not os.path.exists(query_img_path):
124             print(f"错误：待检索图像文件不存在_{query_img_path}")
125             continue
126         print(f"\n{'='*_50}")
127         print(f"Processing_query_image:_{query_img_path}")
128         print(f"{'='*_50}")
129
130     print("Starting_batch_feature_extraction...")
131     batch_extract_resnet50(DATASET_DIR, FEAT_DIRS['ResNet50'])
132     batch_extract_vgg16(DATASET_DIR, FEAT_DIRS['VGG16'])
133     batch_extract_inception(DATASET_DIR, FEAT_DIRS['InceptionV3'])

```

```

133 print("Batch feature extraction completed!")
134 for query_idx in range(QUERY_IMG_COUNT):
135     query_img_path = os.path.join(QUERY_IMG_DIR, f"{query_idx+1}.jpg")
136     print(f"\n{'='*50}")
137     print(f"Processing query image: {query_img_path}")
138     print(f"{'='*50}")
139     query_feat_paths = {
140         'ResNet50': os.path.join(QUERY_FEAT_DIR, f'
141             query_resnet50_{query_idx}.np'),
142         'VGG16': os.path.join(QUERY_FEAT_DIR, f'query_vgg16_{
143             query_idx}.np'),
144         'InceptionV3': os.path.join(QUERY_FEAT_DIR, f'
145             query_inception_{query_idx}.np')
146     }
147     for model_name, feat_dir in FEAT_DIRS.items():
148         print(f"\n--- Querying with {model_name} ---")
149         if model_name == 'ResNet50':
150             test_image = extract_feature.default_loader(
151                 query_img_path)
152             input_image = extract_feature.trans(test_image)
153             input_image = torch.unsqueeze(input_image, 0)
154             image_feature = extract_feature.features(input_image)
155             image_feature = image_feature.detach().numpy()
156             np.save(query_feat_paths[model_name],
157                     image_feature)
158         elif model_name == 'VGG16':
159             from vgg16 import process_image_and_save_features
160             process_image_and_save_features(
161                 query_img_path, query_feat_paths[model_name])
162         elif model_name == 'InceptionV3':
163             from inception import process_and_save_features
164             process_and_save_features(query_img_path,
165                                     query_feat_paths[model_name])
166     top_indices, top_sims = find_top_similar(
167         query_feat_paths[model_name], feat_dir, top_k=5)
168     print(f"Top 5 similar images using {model_name}:")
169     for idx, (img_idx, sim) in enumerate(zip(top_indices,

```

```

        top_sims)):
    print(f"Top_{idx+1}: Index={img_idx}, Similarity={sim:.2f}%")
    result_save_path = os.path.join(OUTPUT_DIR, f'{model_name}_{query_idx}.png')
    plot_similar_results(query_img_path, top_indices, top_sims, model_name, result_save_path)
    print(f"Result plot saved to: {result_save_path}")

if __name__ == '__main__':
    main()

```

3 实验结果及分析

3.1 CIFAR-10 数据集分类

3.1.1 核心结果

分阶段学习率与 30 轮训练的优化策略，ResNet20 模型在 CIFAR-10 测试集上的最终准确率达到 86% 以上，其中最高准确率为 86.21%，满足图像分类任务的性能要求。

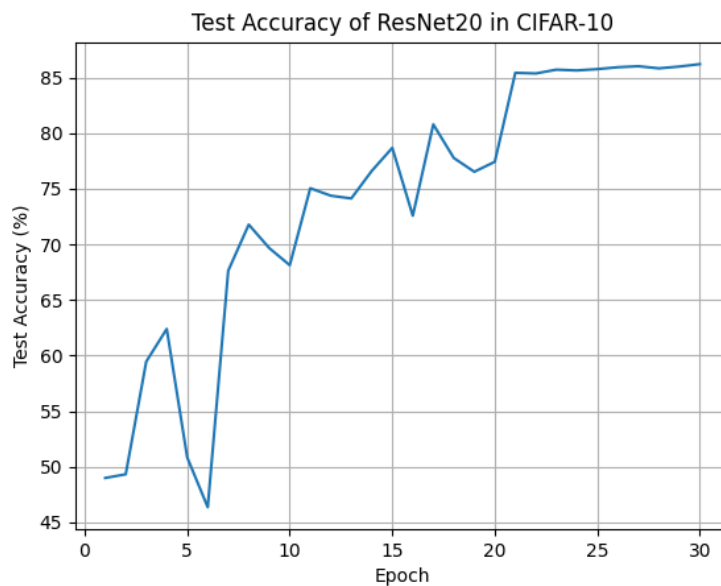


图 1: ResNet20 模型在 CIFAR-10 测试集上的准确率变化曲线

3.1.2 结果分析

结合测试准确率的变化趋势（如图所示），可观察到学习率对模型性能的显著影响：

- 当学习率为 0.1 时，模型收敛速度较快，训练 20 轮后测试准确率已达到 80% 左右，但准确率波动较大。这是因为较大的学习率虽能加速模型向最优解靠近，但易跳过全局最优解陷入局部最优，导致性能不稳定；

- 当学习率调整为 0.01 后，模型进入稳定收敛阶段：在后续 10 轮训练中，测试准确率逐步提升并趋于平稳，最终达到 86.21%。较小的学习率虽减缓了收敛速度，但能更精细地调整模型参数，帮助模型找到更优的参数组合，提升泛化能力。

3.2 实验 5.2 使用 Pytorch 提取图像特征并检索

3.2.1 核心检索结果

实验针对 Query_image 目录中的 3 张待检索图像，分别通过三种模型完成检索，Top-5 相似图像的夹角结果如下（夹角值越小，相似度越高）：

1. 待检索图像 1：



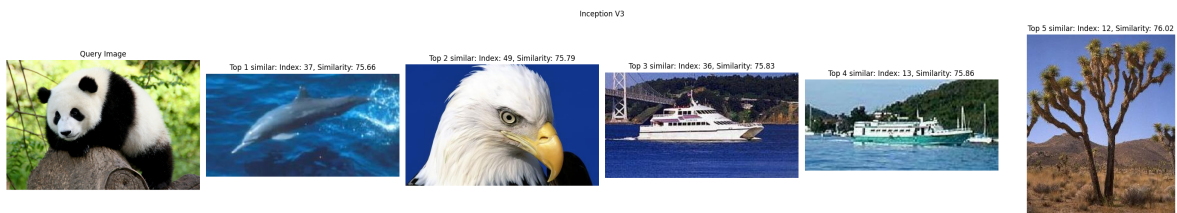
- ResNet50: 5.12, 5.18, 5.19, 5.21, 5.22;



- VGG16: 77.75, 78.83, 78.98, 80.34, 81.55;



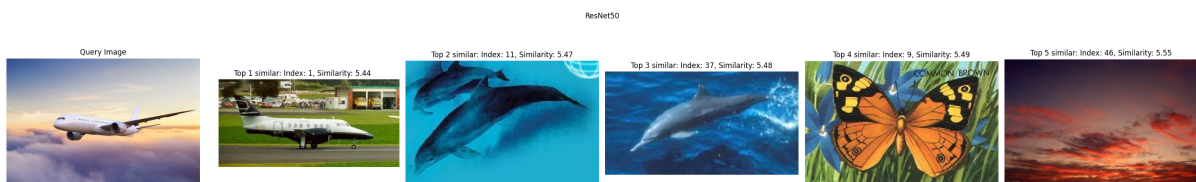
- InceptionV3: 75.57, 75.58, 75.68, 75.72, 76.13;



2. 待检索图像 2:



- ResNet50: 5.44, 5.47, 5.48, 5.49, 5.55;



- VGG16: 73.49, 79.05, 79.32, 80.32, 80.84;

VGG16



- InceptionV3: 75.42, 75.42, 75.75, 75.79, 75.90;

Inception V3



3. 待检索图像 3:

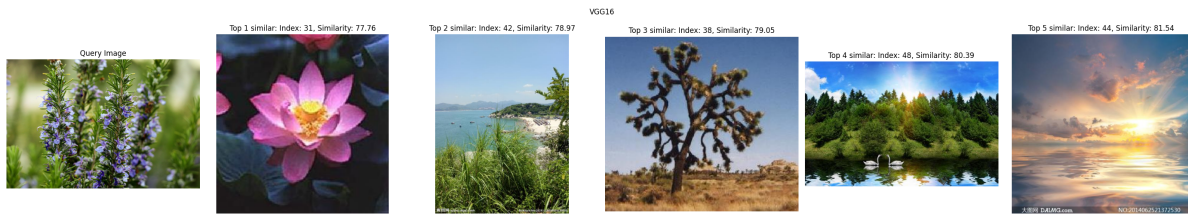


- ResNet50: 5.14, 5.16, 5.22, 5.22, 5.22;

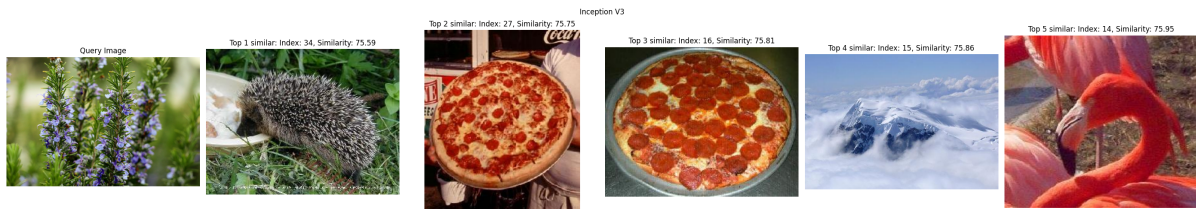
ResNet50



- VGG16: 77.76, 78.97, 79.05, 80.39, 81.54;



- InceptionV3: 75.59, 75.75, 75.82, 75.86, 75.95。



3.2.2 结果分析

从检索结果可得出以下关键结论：

1. 模型性能差异：ResNet50 的检索效果最优，其返回的 Top-5 图像夹角值均小于 6，远低于 VGG16（73 以上）与 InceptionV3（75 以上），说明 ResNet50 提取的特征向量更能反映图像的本质特征，相似度区分度更高；VGG16 虽夹角值较大，但仍能稳定返回相似图像；InceptionV3 的检索效果相对较差，难以匹配出高相似度图像，推测与该模型结构较复杂、特征提取维度与实验数据适配度较低有关；

2. 模型稳定性：在待检索图像 3 的检索任务中，ResNet50 与 InceptionV3 返回的最相似图像夹角值均高于前两张图像的检索结果，说明模型性能受待检索图像内容影响，存在一定不稳定性，后续可通过扩大数据库规模、优化特征后处理方法进一步提升稳定性。

4 实验总结

本次实验通过 Pytorch 框架完成了图像分类与检索两大核心任务，实现了从模型搭建、策略优化到结果分析的完整流程：

1. 分类任务成果：在 CIFAR-10 分类任务中，通过分阶段调整学习率（0.1→0.01）与控制 30 轮训练，使 ResNet20 模型的测试准确率达到 86% 以上，验证了学习率优化对模型性能的关键作用；

2. 检索任务成果：在图像检索任务中，成功利用三种预训练模型完成特征提取与相似度计算，明确了 ResNet50 在该任务中的优势，实现了检索结果的可视化呈现；

3. 知识与技能收获：通过实验深入理解了深度学习模型的训练策略（如学习率调整）、图像特征提取的原理，掌握了利用 Pytorch 框架实现图像分类与检索的核心方法，为后续更复杂的计算机视觉任务奠定了实践基础。

5 思考及感想

5.1 思考题解答

5.1.1 Train acc 与 Test acc 的关联与差异

- 关联：两者均为评估模型性能的核心指标，均通过计算模型预测正确的样本比例得到；在理想训练过程中，两者均会随训练轮次增加呈上升趋势，最终趋于稳定，且 Test acc 的上限受 Train acc 影响。

- 差异：Train acc 是模型在训练集上的准确率，反映模型对训练数据的拟合能力；Test acc 是模型在测试集（未参与训练的数据）上的准确率，反映模型的泛化能力。通常情况下，Train acc 会略高于 Test acc，因模型可能对训练数据存在一定过拟合。

5.1.2 学习率从 0.1 调整为 0.01 后的 acc 变化及原因

学习率调整后，模型的 Test acc 从波动较大逐步趋于稳定，并最终提升至 86.21%。原因在于：

- 学习率为 0.1 时，模型参数更新步长较大，收敛速度快，能快速接近最优解，但易跳过全局最优解陷入局部最优，导致 Test acc 波动；

- 学习率调整为 0.01 时，参数更新步长减小，模型能更精细地搜索最优参数空间，避免局部最优陷阱，泛化能力提升，Test acc 逐步稳定并达到更高水平。

5.2 实验感想

通过本次实验，不仅掌握了 Pytorch 框架在图像任务中的应用方法，更深刻理解了深度学习模型的优化逻辑：训练策略的细微调整（如学习率）可能对模型性能产生显著影响，模型选择需结合具体任务场景（如 ResNet50 更适合图像检索）。同时，实验过程中也认识到模型性能的局限性（如 InceptionV3 的不稳定性），后续需通过数据增强、模型融合等方法进一步优化。本次实验将理论知识与实践操作深度结合，有效提升了对计算机视觉技术的理解与应用能力。